

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

«Запросы, представления и работа с индексами»

Обучающийся: Лютый Никита Артемович

Факультет: Прикладной информатики

Группа: К3240

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург

2026

Содержание

1	Цель работы	2
2	Запросы на выборку данных (Индивидуальное задание)	2
2.1	Запрос 1: Самый активный водитель на конкретной улице	2
2.2	Запрос 2: Авто с пробегом > 250к и без ТО в этом году	2
2.3	Запрос 3: Сколько раз каждый пассажир ездил	3
2.4	Запрос 4: Самый активный пассажир	3
2.5	Запрос 5: Водитель на самой дорогой машине	3
2.6	Запрос 6: Пассажир, который ездит только с одним водителем	3
2.7	Запрос 7: Пробег больше среднего по марке	4
3	Представления	4
3.1	Представление 1: Свободные водители	4
3.2	Представление 2: Зарплата за вчерашний день	4
4	Модификация данных с использованием сложных подзапросов	6
4.1	Операция INSERT	6
4.2	Операция UPDATE	6
4.3	Операция DELETE	7
5	Работа с индексами и EXPLAIN	8
5.1	Анализ производительности простого индекса	8
5.2	Анализ производительности составного индекса	9
5.3	Удаление индексов	9
6	Выводы	10

1 Цель работы

Овладение навыками создания представлений, сложных запросов на выборку и модификацию данных с использованием подзапросов, а также оптимизация производительности БД с помощью индексов.

2 Запросы на выборку данных (Индивидуальное задание)

Ниже приведены реализации запросов согласно варианту №14.

2.1 Запрос 1: Самый активный водитель на конкретной улице

Формулировка: Вывести данные о водителе, который чаще всех доставляет пассажиров на заданную улицу.

```
SELECT s.full_name, COUNT(*) as trip_count
FROM taxi.staff s
JOIN taxi.orders o ON s.staff_id = o.driver_id
WHERE o.to_address LIKE 'ул%. Мира 1%'
GROUP BY s.full_name
HAVING COUNT(*) = (
    SELECT MAX(cnt)
    FROM (
        SELECT COUNT(*) as cnt
        FROM taxi.orders
        WHERE to_address LIKE 'ул%. Мира 1%'
        GROUP BY driver_id
    )
);
```

	full_name character varying (255)	count bigint
1	Быков Д.С.	2

2.2 Запрос 2: Авто с пробегом > 250к и без ТО в этом году

Формулировка: Вывести данные об автомобилях, которые имеют пробег более 250 тысяч километров и которые не проходили ТО в текущем году.

```
SELECT model_mark, plate_number FROM taxi.cars
WHERE mileage > 250000 AND EXTRACT(YEAR FROM last_service_date) < 2026;
```

	model_mark character varying (100)	plate_number character varying (15)
1	Toyota Camry	B111BB78

2.3 Запрос 3: Сколько раз каждый пассажир ездил

Формулировка: Сколько раз каждый пассажир воспользовался услугами таксопарка?

```
SELECT cl.full_name, COUNT(o.order_id)
FROM taxi.clients cl
LEFT JOIN taxi.orders o ON cl.client_id = o.client_id
GROUP BY cl.full_name;
```

	full_name character varying (255)	count bigint
1	Васильев В.В.	1
2	Иванов И.И.	0
3	Сидоров С.С.	4

2.4 Запрос 4: Самый активный пассажир

Формулировка: Вывести данные пассажира, который воспользовался услугами таксопарка максимальное число раз.

```
SELECT * FROM taxi.clients
WHERE client_id IN (
    SELECT client_id
    FROM taxi.orders
    GROUP BY client_id
    HAVING COUNT(*) = (
        SELECT MAX(trip_count)
        FROM (SELECT COUNT(*) as trip_count FROM taxi.orders GROUP BY
client_id)
    )
);
```

	client_id [PK] integer	full_name character varying (255)	phone_number character varying (20)	bank_card character varying (20)
1	1	Сидоров С.С.	+79112223344	1111222233334444

2.5 Запрос 5: Водитель на самой дорогой машине

Формулировка: Вывести данные о водителе, который ездит на самом дорогом автомобиле.

```
SELECT *
FROM taxi.staff s
JOIN taxi.orders o ON s.staff_id = o.driver_id
JOIN taxi.cars c ON o.car_id = c.car_id
WHERE c.cost = (SELECT MAX(cost) FROM taxi.cars);
```

	staff_id integer	full_name character varying (255)	address text	phone_number character varying (20)	job_title character varying (100)	category character varying (10)
1	2	Быков Д.С.	[null]	+79005556677	Водитель	1 класс
2	3	Григорьев Г.Г.	[null]	+79778889900	Водитель	2 класс

2.6 Запрос 6: Пассажир, который ездит только с одним водителем

Формулировка: Вывести данные пассажира, который всегда ездит с одним и тем же водителем.

```

SELECT cl.full_name, cl.phone_number, cl.bank_card
FROM taxi.clients cl
JOIN taxi.orders o ON cl.client_id = o.client_id
GROUP BY cl.client_id, cl.full_name
HAVING COUNT(DISTINCT o.driver_id) = 1;

```

	full_name character varying (255) 🔒	phone_number character varying (20) 🔒	bank_card character varying (20) 🔒
1	Сидоров С.С.	+79112223344	1111222233334444
2	Васильев В.В.	+79223334455	5555666677778888

2.7 Запрос 7: Пробег больше среднего по марке

Формулировка: Какие автомобили имеют пробег больше среднего пробега для своей марки.

```

SELECT model_mark, plate_number, mileage
FROM taxi.cars c1
WHERE mileage > (
    SELECT AVG(mileage)
    FROM taxi.cars c2
    WHERE c2.model_mark = c1.model_mark
);

```

	model_mark character varying (100) 🔒	plate_number character varying (15) 🔒	mileage integer 🔒
1	Toyota Camry	B111BB78	260000

3 Представления

3.1 Представление 1: Свободные водители

Формулировка: сведения о незанятых на данный момент водителях.

```

CREATE OR REPLACE VIEW taxi.v_free_drivers AS
SELECT full_name, phone_number
FROM taxi.staff
WHERE job_title = 'Водитель'
AND NOT EXISTS (
    SELECT 1 FROM taxi.orders
    WHERE taxi.orders.driver_id = taxi.staff.staff_id
    AND taxi.orders.dropoff_time IS NULL
);

```

	full_name character varying (255) 🔒	phone_number character varying (20) 🔒
1	Быков Д.С.	+79005556677
2	Григорьев Г.Г.	+79778889900

3.2 Представление 2: Зарплата за вчерашний день

Формулировка: зарплата всех водителей за вчерашний день.

```
CREATE OR REPLACE VIEW taxi.v_yesterday_salaries AS
SELECT s.full_name, SUM(o.cost * 0.5) as salary_amount
FROM taxi.staff s
JOIN taxi.orders o ON s.staff_id = o.driver_id
WHERE o.order_date = CURRENT_DATE - INTERVAL '1 day'
GROUP BY s.full_name;
```

	full_name character varying (255) 🔒	salary_amount numeric 🔒
1	Быков Д.С.	350.000

4 Модификация данных с использованием сложных подзапросов

4.1 Операция INSERT

Задача: Начислить премию 15 процентов от общей суммы выручки водителям, чье количество заказов выше среднего по таксопарку.

До модификации:

	salary_id [PK] integer	staff_id integer	calc_date date	total_amount numeric (15,2)
1	1	2	2026-03-12	40000.00
2	2	3	2026-03-12	40000.00

```
INSERT INTO taxi.salary (staff_id, calc_date, total_amount)
SELECT driver_id, CURRENT_DATE, SUM(cost) * 0.15
FROM taxi.orders
WHERE driver_id IS NOT NULL
GROUP BY driver_id
HAVING COUNT(order_id) > (SELECT AVG(order_count) FROM (SELECT COUNT(
order_id) as order_count FROM taxi.orders GROUP BY driver_id) as
avg_table);
```

После модификации:

	salary_id [PK] integer	staff_id integer	calc_date date	total_amount numeric (15,2)
1	1	2	2026-03-12	40000.00
2	2	3	2026-03-12	40000.00
3	3	2	2026-04-12	555.00

4.2 Операция UPDATE

Задача: Снизить стоимость всех заказов на 15 процентов для тех клиентов, чье общее количество поездок превышает среднее количество поездок на одного клиента.

До модификации:

	client_id integer	from_address text	cost numeric (12,2)
1	1	ул. Ленина 1	500.00
2	1	Дом	700.00
3	1	Аэропорт	2000.00
4	1	ул. Лермонтова 2	500.00
5	2	ул. Садовая 1	300.00

```
UPDATE taxi.orders
SET cost = cost * 0.85
WHERE client_id IN (
    SELECT client_id
    FROM taxi.orders
    GROUP BY client_id
    HAVING COUNT(order_id) > (
        SELECT AVG(order_count)
        FROM (SELECT COUNT(order_id) as order_count FROM taxi.orders
        GROUP BY client_id) as sub
    )
);
```

После модификации:

	client_id integer	from_address text	cost numeric (12,2)
1	2	ул. Садовая 1	300.00
2	1	ул. Ленина 1	425.00
3	1	Дом	595.00
4	1	Аэропорт	1700.00
5	1	ул. Лермонтова 2	425.00

4.3 Операция DELETE

Задача: Удалить записи из графика работы для тех водителей, чья общая выручка по заказам ниже средней выручки на одного водителя.

До модификации:

	schedule_id integer	full_name character varying (255)	work_date date
1	1	Быков Д.С.	2026-04-12
2	2	Григорьев Г.Г.	2026-04-12

```
DELETE FROM taxi.work_schedule
WHERE staff_id IN (
    SELECT driver_id
    FROM taxi.orders
    GROUP BY driver_id
    HAVING SUM(cost) < (
        SELECT AVG(total_revenue)
        FROM (SELECT SUM(cost) as total_revenue FROM taxi.orders GROUP
BY driver_id) as sub
    )
);
```

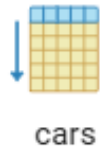
После модификации:

	schedule_id integer	full_name character varying (255)	work_date date
1	1	Быков Д.С.	2026-04-12

5 Работа с индексами и EXPLAIN

5.1 Анализ производительности простого индекса

Проведено сравнение времени выполнения запроса до и после создания индекса.



	QUERY PLAN text
1	Seq Scan on cars (cost=0.00..12.00 rows=53 width=454) (actual time=0.018..0.020 rows=1 loops=...
2	Filter: (mileage > 200000)
3	Rows Removed by Filter: 2
4	Planning Time: 0.098 ms
5	Execution Time: 0.037 ms

Рис. 1: Графический план запроса до создания индекса в pgAdmin 4



	QUERY PLAN text
1	Index Scan using idx_cars_mileage on cars (cost=0.13..8.15 rows=1 width=454) (actual time=0.016..0.017 rows=1 loops=...
2	Index Cond: (mileage > 200000)
3	Planning Time: 0.097 ms
4	Execution Time: 0.032 ms

Рис. 2: Графический план запроса после создания индекса в pgAdmin 4

```
EXPLAIN ANALYZE
SELECT * FROM taxi.cars WHERE mileage > 200000;

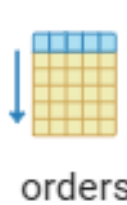
CREATE INDEX idx_cars_mileage_btree ON taxi.cars (mileage);

EXPLAIN ANALYZE
SELECT * FROM taxi.cars WHERE mileage > 200000;
```

1. **Без индекса:** Использовался Seq Scan. Время: 0.037 ms.
2. **С индексом:** Использовался Index Scan. Время: 0.032 ms.

5.2 Анализ производительности составного индекса

Проведено сравнение времени выполнения запроса до и после создания индекса.



QUERY PLAN	
text	
1	Seq Scan on orders (cost=0.00..14.65 rows=1 width=234) (actual time=0.022..0.024 rows=1 loops=...
2	Filter: ((cost > '500'::numeric) AND (to_address = 'ул. Мира 1'::text))
3	Rows Removed by Filter: 4
4	Planning Time: 0.102 ms
5	Execution Time: 0.040 ms

Рис. 3: Графический план запроса до создания индекса в pgAdmin 4



QUERY PLAN	
text	
1	Index Scan using idx_order_addr_cost on orders (cost=0.13..8.15 rows=1 width=234) (actual time=0.026..0.027 rows=1 loops=...
2	Index Cond: ((to_address = 'ул. Мира 1'::text) AND (cost > '500'::numeric))
3	Planning Time: 0.083 ms
4	Execution Time: 0.039 ms

Рис. 4: Графический план запроса после создания индекса в pgAdmin 4

```
EXPLAIN ANALYZE
SELECT * FROM taxi.orders WHERE to_address = 'ул. Мира 1' AND cost > 500;

CREATE INDEX idx_orders_complex ON taxi.orders (to_address, cost);

EXPLAIN ANALYZE
SELECT * FROM taxi.orders WHERE to_address = 'ул. Мира 1' AND cost > 500;
```

1. **Без индекса:** Использовался Seq Scan. Время: 0.040 ms.
2. **С индексом:** Использовался Index Scan. Время: 0.039 ms.

5.3 Удаление индексов

```
DROP INDEX taxi.idx_cars_mileage;
DROP INDEX taxi.idx_orders_addr_cost;
```

6 Выводы

В ходе работы реализованы сложные SQL-запросы, соответствующие бизнес-логике таксопарка. Было измерено отличие использования индексов от обычного обхода таблицы, что позволило сократить время выполнения выборки.